

# 进程同步实验（生产者-消费者问题）

基于openEuler 操作系统，设计一个C语言程序，利用信号量机制解决有限 缓冲的生产者-消费者问题。

## 实验内容

具体内容参考教材《操作系统概念》第六章项目：生产者-消费者问题。

## 实验要求

### (1) 缓冲区

(a) 缓冲区存储结构建议采用固定大小的数组表示，并作为环形队列处理。

(b) 缓冲区的访问算法按照课本6.6.1节图6.10、图6.11进行设计。

### (2) 主函数main()

(a) 主函数需要创建一定数量的生产者线程与消费者线程。线程创建完毕后，主函数将睡眠一段时间，并在唤醒时终止应用程序。

(b) 主函数需要从命令行接受三个参数：睡眠时长、生产者线程数量、消费者线程数量。

### (3) 生产者与消费者线程

(a) 生产者线程：随机睡眠一段时间，向缓冲区插入一个随机数。

(b) 消费者线程：随机睡眠一段时间，从缓冲区去除一个随机数。

## 程序设计说明

### 1. 缓冲区设计

```
1  #define BUFFER_SIZE 5 // 缓冲区大小
2
3  int buffer[BUFFER_SIZE];
4  int in = 0; // 指向下一个插入位置
5  int out = 0; // 指向下一个取出位置
6  void insert_item(int item)
7  {
8      buffer[in] = item;
9      in = (in + 1) % BUFFER_SIZE;
10 }
11
12 int remove_item()
13 {
14     int item = buffer[out];
15     out = (out + 1) % BUFFER_SIZE;
16     return item;
17 }
```

- 缓冲区使用了一个固定大小的**数组**（buffer），默认大小是5，并将其逻辑上作为一个**环形队列**。
- 指针 `in` 和 `out` 分别指向缓冲区的插入位置和取出位置，使用模运算实现环形队列的循环。

## 2. 线程同步机制

```
1  sem_t empty;           // 表示空缓冲区单元的信号量
2  sem_t full;            // 表示已占用缓冲区单元的信号量
3  pthread_mutex_t mutex; // 保护缓冲区的互斥量
```

Fence 2

- 使用信号量管理缓冲区的状态：
  - `empty`：表示缓冲区中空单元的数量（初始值为缓冲区大小）。
  - `full`：表示缓冲区中已占用单元的数量（初始值为0）。
- 使用**互斥量**（`pthread_mutex_t`）确保生产者和消费者对缓冲区的访问互斥，避免同时修改缓冲区数据。

## 3. 生产者线程

```
1  void *producer(void *param)
2  {
3      int id = *((int *)param); // 转换参数为整数ID
4      int item;
5      unsigned int seed = time(NULL) ^ pthread_self();
6      srand(seed);
7      while (1)
8      {
9          sleep(rand() % 3);
10
11         item = rand() % 1000;
12
13         sem_wait(&empty);
14         pthread_mutex_lock(&mutex);
15
16         insert_item(item);
17         printf("Producer %d produced +++ %d\n", id, item);
18
19         pthread_mutex_unlock(&mutex);
20         sem_post(&full);
21     }
22     return NULL;
23 }
```

Fence 3

- 生产者线程随机休眠一段时间后，生成一个随机数。
- 线程依次执行以下操作：
  1. 调用 `sem_wait(&empty)` 检查是否有空单元，若无则阻塞等待。

2. 获取互斥锁 `pthread_mutex_lock(&mutex)`，向缓冲区插入数据。
3. 打印生产信息，并释放互斥锁 `pthread_mutex_unlock(&mutex)`。
4. 调用 `sem_post(&full)` 通知消费者有新数据可用。

---

## 4. 消费者线程

```
1 void *consumer(void *param)
2 {
3     int id = *((int *)param); // 转换参数为整数ID
4     int item;
5     while (1)
6     {
7         sleep(rand() % 3); // 随机睡眠0-2秒
8
9         sem_wait(&full);
10        pthread_mutex_lock(&mutex);
11
12        item = remove_item();
13        printf("Consumer %ld consumed --- %d\n", id, item);
14
15        pthread_mutex_unlock(&mutex);
16        sem_post(&empty);
17    }
18    return NULL;
19 }
```

Fence 4

- 消费者线程随机休眠一段时间后，依次执行以下操作：
  1. 调用 `sem_wait(&full)` 检查是否有可用单元，若无则阻塞等待。
  2. 获取互斥锁 `pthread_mutex_lock(&mutex)`，从缓冲区移除数据。
  3. 打印消费信息，并释放互斥锁 `pthread_mutex_unlock(&mutex)`。
  4. 调用 `sem_post(&empty)` 通知生产者有空单元可用。

---

## 5. 主函数

```
1 int main(int argc, char *argv[])
2 {
3     srand(time(NULL));
4     if (argc != 4)
5     {
6         fprintf(stderr, "Usage: %s <sleep_time> <producer_count>
7         <consumer_count>\n", argv[0]);
8         return -1;
9     }
10    int sleep_time = atoi(argv[1]);
11    int producer_count = atoi(argv[2]);
12    int consumer_count = atoi(argv[3]);
```

```
13 pthread_t producers[producer_count];
14 pthread_t consumers[consumer_count];
15
16 // 初始化信号量和互斥量, 检查返回值
17 if (sem_init(&empty, 0, BUFFER_SIZE) == -1 ||
18     sem_init(&full, 0, 0) == -1 ||
19     pthread_mutex_init(&mutex, NULL) != 0)
20 {
21     perror("Init failed");
22     return -1;
23 }
24
25 // 创建生产者线程
26 // 在main函数中创建线程时
27 int producer_ids[producer_count];
28 for (int i = 0; i < producer_count; i++)
29 {
30     producer_ids[i] = i + 1; // 生成1~n的编号
31     if (pthread_create(&producers[i], NULL, producer, &producer_ids[i]) !=
32         0)
33     {
34         perror("Failed to create producer thread");
35         return -1;
36     }
37
38 // 创建消费者线程
39 int consumer_ids[consumer_count];
40 for (int i = 0; i < consumer_count; i++)
41 {
42     consumer_ids[i] = i + 1; // 生成1~n的编号
43     if (pthread_create(&consumers[i], NULL, consumer, &consumer_ids[i]) !=
44         0)
45     {
46         perror("Failed to create consumer thread");
47         return -1;
48     }
49
50 // 主线程睡眠指定时间
51 sleep(sleep_time);
52
53 // 清理资源
54 for (int i = 0; i < producer_count; i++)
55 {
56     pthread_cancel(producers[i]);
57     pthread_join(producers[i], NULL);
58 }
59
60 for (int i = 0; i < consumer_count; i++)
61 {
62     pthread_cancel(consumers[i]);
63     pthread_join(consumers[i], NULL);
64 }
65
66 // 销毁信号量和互斥量
```

```

67     sem_destroy(&empty);
68     sem_destroy(&full);
69     pthread_mutex_destroy(&mutex);
70
71     printf("Main thread exiting.\n");
72     return 0;
73 }

```

Fence 5

1. **参数解析**：从命令行读取睡眠时长、生产者线程数量、消费者线程数量。
2. **信号量和互斥量初始化**：确保程序启动时信号量和互斥量处于正确状态。
3. **创建线程**：
  - 为每个生产者和消费者分配一个唯一的 ID，并创建对应线程。
  - 如果线程创建失败，立即终止程序并打印错误信息。
4. **主线程睡眠**：模拟程序运行的时长。
5. **线程清理**：使用 `pthread_cancel` 和 `pthread_join` 停止并清理线程。
6. **资源销毁**：程序结束前销毁信号量和互斥量。

## 编译运行(假设文件存为producer\_consumer.c)

```

1 gcc -static -pthread -o producer_consumer producer_consumer.c -lpthread
2 ./producer_consumer 5 5 5

```

Fence 6

Usage: ./producer\_consumer <sleep\_time> <producer\_count> <consumer\_count>

## 测试报告

### windows

- 1.

```
PS D:\codefeild\code_c\project3> gcc -static -pthread -o producer_consumer producer_consumer.c -lpthread
PS D:\codefeild\code_c\project3> ./producer_consumer 10 2 2
Producer 1 produced +++ 373
Producer 2 produced +++ 895
Producer 1 produced +++ 915
Producer 2 produced +++ 261
Consumer 1 consumed --- 373
Consumer 2 consumed --- 895
Producer 1 produced +++ 584
Producer 2 produced +++ 804
Producer 2 produced +++ 16
Consumer 2 consumed --- 915
Consumer 1 consumed --- 261
Producer 1 produced +++ 2
Producer 2 produced +++ 311
Consumer 2 consumed --- 584
Consumer 1 consumed --- 804
Producer 2 produced +++ 104
Producer 1 produced +++ 635
Consumer 2 consumed --- 16
Consumer 1 consumed --- 2
Producer 1 produced +++ 237
Producer 2 produced +++ 777
Consumer 1 consumed --- 311
Consumer 2 consumed --- 104
Producer 2 produced +++ 175
Producer 1 produced +++ 880
Consumer 2 consumed --- 635
Consumer 2 consumed --- 237
Consumer 2 consumed --- 777
Producer 1 produced +++ 653
Consumer 1 consumed --- 175
Consumer 1 consumed --- 880
Consumer 1 consumed --- 653
Producer 2 produced +++ 556
Consumer 2 consumed --- 556
Main thread exiting.
```

Figure 1

分析说明：测试了10秒，两个消费者两个生产者，程序正常运行结束，没有发生死锁

2.

```
PS D:\codefeild\code_c\project3> ./producer_consumer 5 5 5
Producer 2 produced +++ 802
Producer 3 produced +++ 551
Producer 3 produced +++ 357
Producer 4 produced +++ 848
Producer 5 produced +++ 596
Consumer 2 consumed --- 802
Producer 2 produced +++ 61
Consumer 5 consumed --- 551
Consumer 1 consumed --- 357
Consumer 3 consumed --- 848
Consumer 4 consumed --- 596
Producer 3 produced +++ 752
Producer 3 produced +++ 274
Producer 3 produced +++ 264
Producer 1 produced +++ 822
Consumer 4 consumed --- 61
Consumer 2 consumed --- 752
Consumer 1 consumed --- 274
Consumer 3 consumed --- 264
Consumer 5 consumed --- 822
Producer 3 produced +++ 175
Producer 5 produced +++ 817
Producer 1 produced +++ 766
Producer 2 produced +++ 593
Producer 4 produced +++ 753
Consumer 4 consumed --- 175
Consumer 5 consumed --- 817
Consumer 3 consumed --- 766
Consumer 1 consumed --- 593
Consumer 2 consumed --- 753
Producer 5 produced +++ 960
Producer 5 produced +++ 69
Producer 4 produced +++ 33
Producer 3 produced +++ 22
Consumer 3 consumed --- 960
Consumer 1 consumed --- 69
Consumer 5 consumed --- 33
Consumer 4 consumed --- 22
Producer 4 produced +++ 997
Consumer 2 consumed --- 997
Main thread exiting.
```

Figure 2

分析说明：测试了5秒5个生产者线程5个消费者线程的情况，没有发生死锁，且缓冲区最多的情况也是5个（缓冲区大小默认是5）

linux

```
zy@debian:~$ ./hello 5 4 1
Producer 4 produced +++ 291
Producer 2 produced +++ 901
Producer 4 produced +++ 397
Producer 2 produced +++ 734
Producer 1 produced +++ 270
Consumer 1 consumed --- 291
Producer 3 produced +++ 56
Consumer 1 consumed --- 901
Producer 3 produced +++ 368
Consumer 1 consumed --- 397
1. Producer 4 produced +++ 404
Consumer 1 consumed --- 734
Producer 1 produced +++ 305
Consumer 1 consumed --- 270
Producer 3 produced +++ 45
Consumer 1 consumed --- 56
Producer 2 produced +++ 398
Consumer 1 consumed --- 368
Producer 4 produced +++ 10
Consumer 1 consumed --- 404
Producer 4 produced +++ 976
Main thread exiting.
```

分析说明：在linux上相同代码也能正常编译运行，这是多个生产者一个消费者的情况，生产者在生产了5个之后使得缓冲区满了，由于消费者线程数少，接下来基本上是交替生产和消费

2.

```
zy@debian:~$ ./hello 10 1 4
Producer 1 produced +++ 405
Consumer 1 consumed --- 405
Producer 1 produced +++ 95
Consumer 3 consumed --- 95
Producer 1 produced +++ 180
Consumer 2 consumed --- 180
Producer 1 produced +++ 469
Consumer 1 consumed --- 469
Producer 1 produced +++ 34
Consumer 4 consumed --- 34
Producer 1 produced +++ 19
Consumer 3 consumed --- 19
Producer 1 produced +++ 381
Consumer 2 consumed --- 381
Main thread exiting.
```

Figure 3

分析说明：4个消费者对一个生产者，生产后马上就被消费了。

**经测试1000秒20个生产者20个消费者也未发生死锁和其他异常，说明了代码基本正确实现了生产者消费者模型**

## 总结

这次实验让我熟悉了信号量在进/线程同步中的使用，以及用信号量解决生产者消费者问题，加强了我对posix api的理解与使用。这次实验，使我对多线程同步、信号量的使用以及C语言多线程编程有了更加深刻的理解，为将来处理类似问题积累了宝贵经验。

遇到的问题

1.pthread库的链接，一开始不会用gcc链接，后面询问gpt解决了

2.随机数的生成，刚开始没给生产者随机数种子，只在main函数中设置了随机数种子，发现每次几个生产者产生的随机数都是一样的

后面了解到rand()可能不是线程安全，通过给每个生产者设置一个随机数种子并与自己的线程id异或，尽量保证了每个线程产生的随机数不一样

可以改进的地方

1.增加缓冲区满或空的提示，这可能需要更加精细的互斥管理，或者调整一部分输出顺序

2.增加实时时间的输出

3.缓冲区改为可以动态设置，而不是固定为5写进代码里